Go 语言笔试面试题(代码输出)

2020-10-10 23:10:10

golang interview questions

Go 语言笔试面试题汇总，Github

常量与变量

1. 下列代码的输出是：

```
func main() {
    const (
        a, b = "golang", 100
        d, e
        f bool = true
        g
    )
    fmt.Println(d, e, g)
}
```

答案

golang 100 true

在同一个 const group 中，如果常量定义与前一行的定义一致，则可以省略类型和值。编译时，会按照前一行的定义自动补全。即等价于

```
func main() {
    const (
        a, b = "golang", 100
```

```
        d, e = "golang", 100
        f bool = true
        g bool = true
    )
    fmt.Println(d, e, g)
}
```

2. 下列代码的输出是：

```
func main() {
    const N = 100
    var x int = N

    const M int32 = 100
    var y int = M
    fmt.Println(x, y)
}
```

答案

编译失败：cannot use M (type int32) as type int in assignment

Go 语言中，常量分为无类型常量和有类型常量两种，`const N = 100`，属于无类型常量，赋值给其他变量时，如果字面量能够转换为对应类型的变量，则赋值成功，例如，`var x int = N`。但是对于有类型的常量 `const M int32 = 100`，赋值给其他变量时，需要类型匹配才能成功，所以显示地类型转换：

```
var y int = int(M)
```

3. 下列代码的输出是：

```
func main() {
    var a int8 = -1
    var b int8 = -128 / a
    fmt.Println(b)
}
```

答案

-128

int8 能表示的数字的范围是 [-2^7, 2^7-1]，即 [-128, 127]。-128 是无类型常量，转换为 int8，再除以变量 -1，结果为 128，常量除以变量，结果是一个变量。变量转换时允许溢出，符号位变为 1，转为补码后恰好等于 -128。

对于有符号整型，最高位是是符号位，计算机用补码表示负数。补码 = 原码取反加一。

例如：

```
-1 :  11111111
00000001(原码)    11111110(取反)    11111111(加一)
-128:
10000000(原码)    01111111(取反)    10000000(加一)

-1 + 1 = 0
11111111 + 00000001 = 00000000(最高位溢出省略)
-128 + 127 = -1
10000000 + 01111111 = 11111111
```

4. 下列代码的输出是：

```go
func main() {
    const a int8 = -1
    var b int8 = -128 / a
    fmt.Println(b)
}
```

答案

编译失败：constant 128 overflows int8

-128 和 a 都是常量，在编译时求值，-128 / a = 128，两个常量相除，结果也是一个常量，常量类型转换时不允许溢出，因而编译失败。

作用域

1. 下列代码的输出是：

```go
func main() {
    var err error
    if err == nil {
        err := fmt.Errorf("err")
        fmt.Println(1, err)
    }
    if err != nil {
        fmt.Println(2, err)
    }
}
```

答案

1 err

`:=` 表示声明并赋值，`=` 表示仅赋值。

变量的作用域是大括号，因此在第一个 if 语句 `if err == nil` 内部重新声明且赋值了与外部变量同名的局部变量 err。对该局部变量的赋值不会影响到外部的 err。因此第二个 if 语句 `if err != nil` 不成立。所以只打印了 `1 err`。

defer 延迟调用

1. 下列代码的输出是：

```
type T struct{}

func (t T) f(n int) T {
    fmt.Print(n)
    return t
}

func main() {
    var t T
    defer t.f(1).f(2)
    fmt.Print(3)
}
```

答案

132

defer 延迟调用时，需要保存函数指针和参数，因此链式调用的情况下，除了最后一个函数/方法外的函数/方法都会在调用时直接执行。也就是说 `t.f(1)` 直接执行，然后执行 `fmt.Print(3)`，最后函数返回时再执行 `.f(2)`，因此输出是 132。

2. 下列代码的输出是：

```
func f(n int) {
    defer fmt.Println(n)
    n += 100
}

func main() {
    f(1)
}
```

答案

1

打印 1 而不是 101。defer 语句执行时，会将需要延迟调用的函数和参数保存起来，也就是说，执行到 defer 时，参数 n(此时等于 1) 已经被保存了。因此后面对 n 的改动并不会影响延迟函数调用的结果。

3. 下列代码的输出是：

```
func main() {
    n := 1
    defer func() {
        fmt.Println(n)
    }()
```

```
    n += 100
}
```

答案

101

匿名函数没有通过传参的方式将 n 传入，因此匿名函数内的 n 和函数外部的 n 是同一个，延迟执行时，已经被改变为 101。

4. 下列代码的输出是：

```
func main() {
    n := 1
    if n == 1 {
        defer fmt.Println(n)
        n += 100
    }
    fmt.Println(n)
}
```

答案

```
101
1
```

先打印 101，再打印 1。defer 的作用域是函数，而不是代码块，因此 if 语句退出时，defer 不会执行，而是等 101 打印后，整个函数返回时，才会执行。